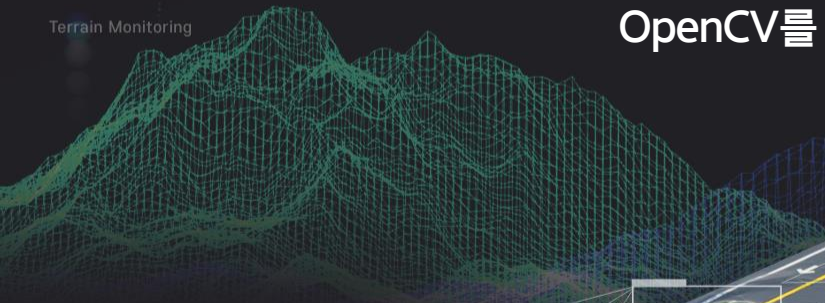


Image Recognition



Terrain Monitoring



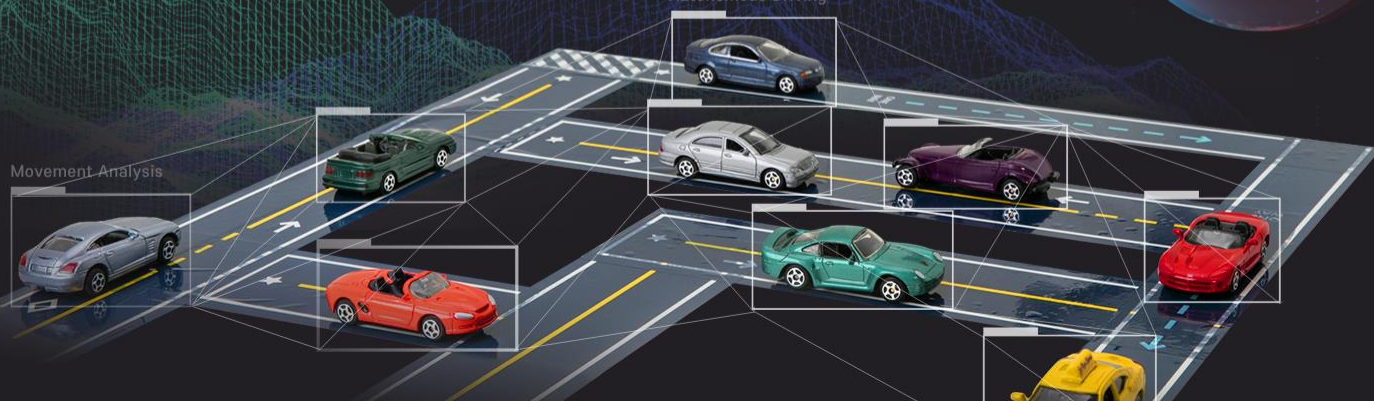
강원혁신플랫폼

컴퓨터 비전

OpenCV를 활용한 영상 정보 전처리(1)

Autonomous Driving

Movement Analysis

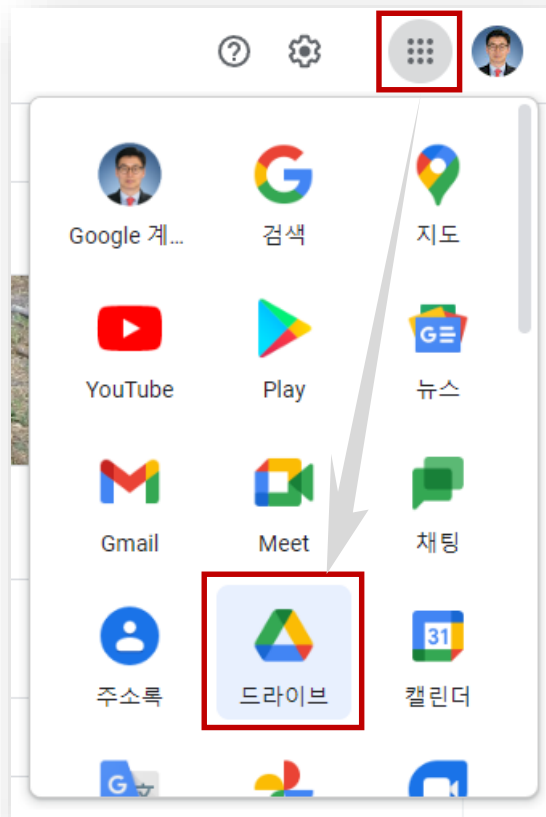




■ 기본 시각지능 기술

먼저, 구글 Colab을 이용해 보자.

- 구글 크롬 브라우저에서 우측 그림과 같이 **구글 드라이브**를 클릭한다.

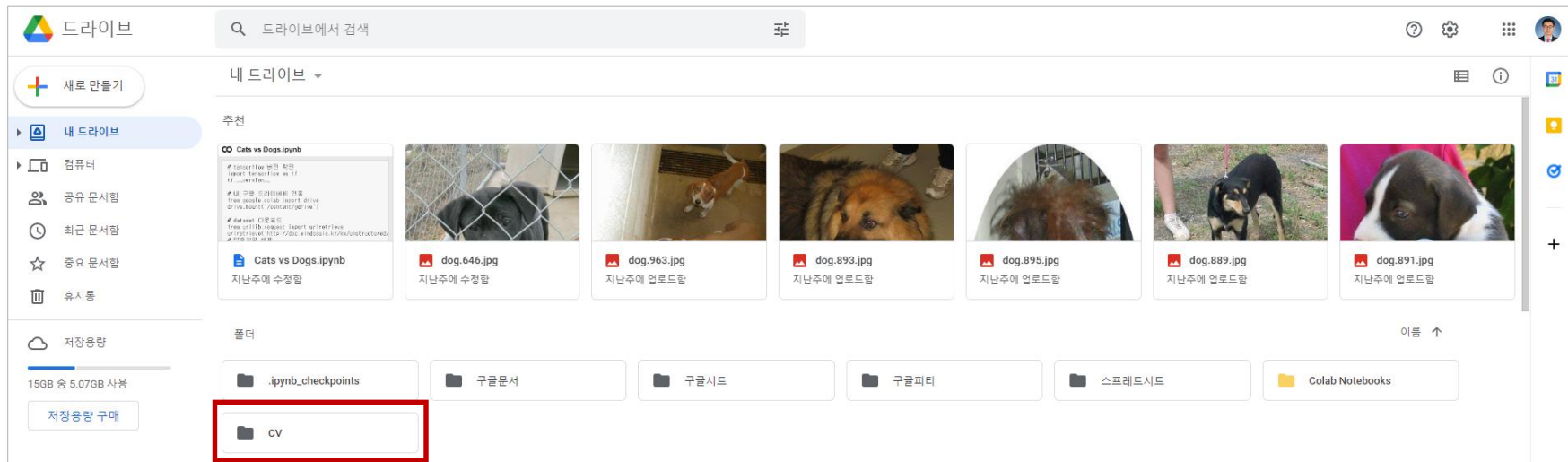




■ 구글 크롬 브라우저에서 아래 그림과 같이 구글 드라이브의 **CV폴더**를 클릭한다.

➡ CV 폴더는 개와 강아지 분류 문제에서 생성된 폴더이다.

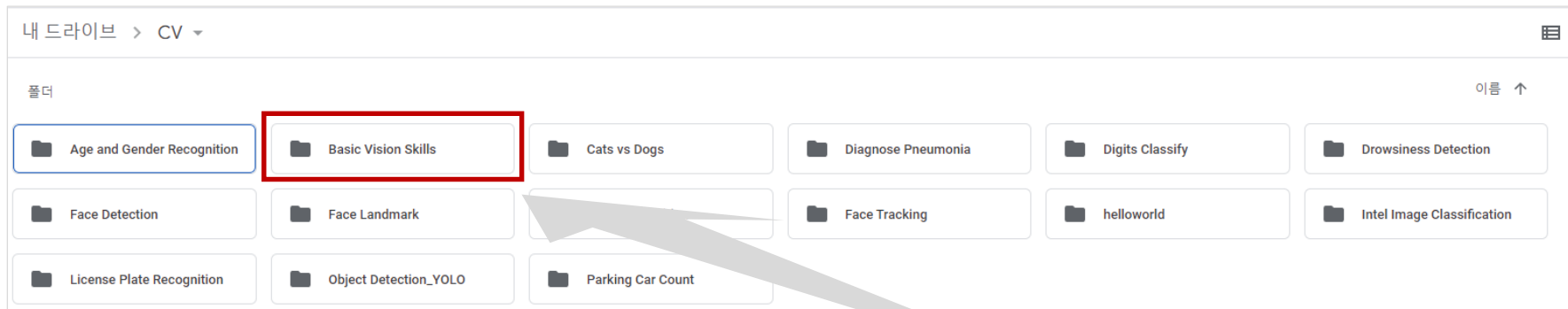
※ CV 폴더가 없는 경우 직접 생성해도 됩니다(CV 폴더가 최상위 폴더).



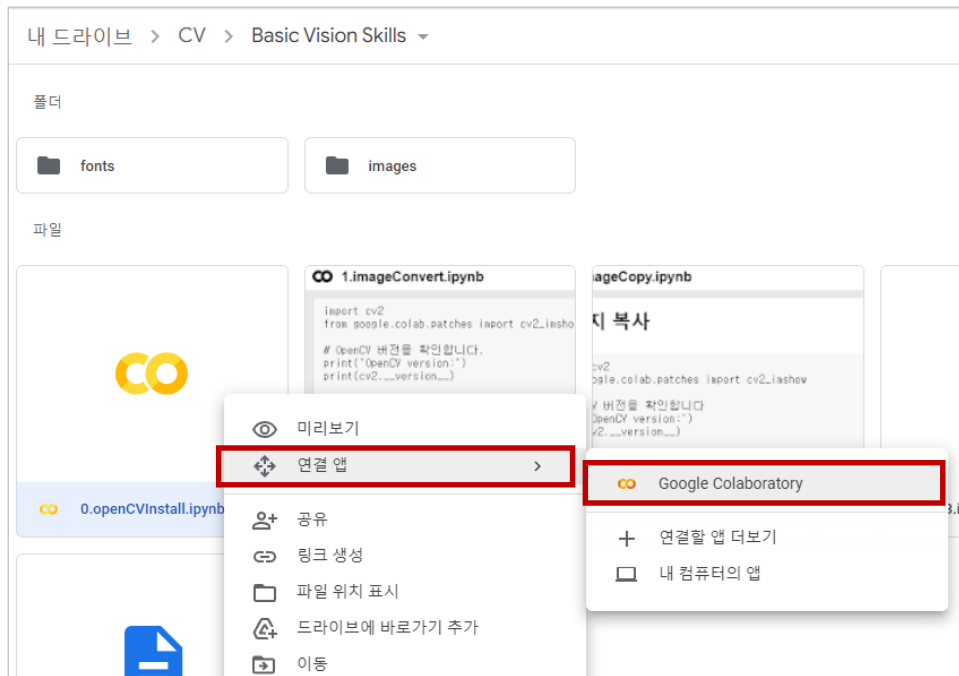


■ 구글 드라이브 > CV > **Basic Vision Skills** 폴더를 클릭한다.

● 강의 실습 자료를 다운로드하여 [Basic Vision Skills] 폴더를
구글 드라이브 CV 폴더 아래에 복사한다.



- 아래의 그림의 Basic Vision Skills 폴더에서 0.openCVInstall.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



■ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.


0.openCVInstall.ipynb


파일 수정 보기 삽입 런타임 도구 도움말 2022년 9월 16일에 마지막으로 수정됨

☰

🔍

{x}

📁

+ 코드

+ 텍스트

▶ # OpenCV 설치하기

[] # 설치된 패키지 확인하기

!pip list

Package	Version
absl-py	1.2.0
aepp1	0.0.33
aesara	2.7.9
aihttp	3.8.1
aiosignal	1.2.0
alabaster	0.7.12

OpenCV(Open Source Computer Vision)

- OpenCV는 이미지를 조작하는 데에 있어 매우 강력한 모듈이다.

OpenCV는 Open Source Computer Vision을 의미하는 이름이며,
실시간 컴퓨터 비전을 위한 라이브러리이다.

- ② 파이썬 프로그래밍 환경에서 OpenCV를 사용하기 위해서는 `opencv-python` 패키지를 설치한다.
- ② OpenCV 이미지 데이터를 NumPy 배열로 저장하지만 색 채널의 순서가 RGB 순서가 아니라, BGR 순서로 뒤바뀌어 있다는 점에 주의한다.
- 이는 OpenCV를 개발하던 초기에는 BGR 색공간이 카메라 제조사나 소프트웨어 개발자들에게 보편적이었기 때문임

Local PC 아나콘다 환경인 경우

- Local PC 환경에서 Jupyter notebook으로 Python을 실행하는 환경이라면 아래와 같은 명령어로 OpenCV를 설치한다.

```
# OpenCV 설치하기
```

```
!pip3 install opencv-python
```

- 🔄 Local PC 환경인 경우 필요한 모듈(라이브러리, 패키지)를 직접 설치한다.

구글 Colab 환경인 경우

- 구글 Colab 환경에서는 실행할 때에는 OpenCV 설치 과정이 필요하지 않다.

tensorboard	2.12.3
tensorboard-data-server	0.7.1
tensorflow	2.12.0
tensorflow-datasets	4.9.2
tensorflow-estimator	2.12.0
tensorflow-gcs-config	2.12.0
tensorflow-hub	0.13.0
tensorflow-io-gcs-filesystem	0.32.0
tensorflow-metadata	1.13.1
tensorflow-probability	0.20.1
tensorstore	0.1.38

- 🔄 구글 Colab은 자주 사용되는 패키지나 프레임워크가 기본적으로 설치된 VM 환경을 제공한다.



■ 0.openCVInstall.ipynb : 현재 환경에 설치된 모듈 정보 확인

➡ 다음은 현재 환경에 설치된 모든 모듈(라이브러리, 패키지)의 버전을 확인해 보자.

➡ pip list는 현재 환경에 설치된 모든 모듈의 버전을 확인해 준다.

```
1 # pip list : 현재 환경에 설치된 모든 모듈(라이브러리, 패키지)의 버전 확인한다.  
2  
3 !pip list
```

opencv-contrib-python	4.7.0.72
opencv-python	4.7.0.72
opencv-python-headless	4.7.0.72
openpyxl	3.0.10
opt-einsum	3.3.0
optax	0.1.5
orbax-checkpoint	0.2.6
osqp	0.6.2.post8
packaging	23.1
palettable	3.3.3
pandas	1.5.3
pandas-datareader	0.10.0
pandas-gbq	0.17.9
pandocfilters	1.5.0
panel	0.14.4

➡ 위 그림과 같이 opencv-python 패키지가 설치된 것을 확인해 볼 수 있다.



■ 다음은 **현재 환경에 개별 모듈**(라이브러리, 패키지)의 **버전을 확인**해 보자.

➡ pip show '모듈명'는 현재 환경에 설치된 개별 모듈의 버전을 확인해 준다.



```
1 # pip show '모듈명' : 개별 모듈(라이브러리, 패키지)의 상세정보를 통하여 버전을 확인
2
3 !pip show 'tensorflow'      # tensorflow 모듈의 버전확인
```



```
Name: tensorflow
Version: 2.12.0
Summary: TensorFlow is an open source machine learning framework for everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: /usr/local/lib/python3.10/dist-packages
Requires: absl-py, astunparse, flatbuffers, gast, google-pasta, grpcio, h5py, jax, keras, libclang, numpy,
Required-by: dopamine-rl
```

➡ 위 그림과 같이 '**tensorflow**' 모듈의 **버전 정보**를 확인할 수 있다.

■ 다음은 **현재 환경에 개별 모듈(라이브러리, 패키지)의 버전을 확인**해 보자.

☞ ‘__version__’는 현재 환경에 설치된 개별 모듈의 버전을 확인해 준다.

```
1  # __version__ : 해당 모듈(라이브러리, 패키지)의 버전을 확인
2
3  import numpy as np
4
5  print('Numpy Version :', np.__version__)
```

Numpy Version : 1.22.4

☞ 위 그림과 같이 ‘**numpy**’ 모듈의 **버전 정보**를 확인할 수 있다.

OpenCV : 이미지 변환

1.imageConvert.ipynb : cv2로 이미지 읽기, 이미지 변환

OpenCV 패키지를 통해 아래의 기능을 수행해 보자.

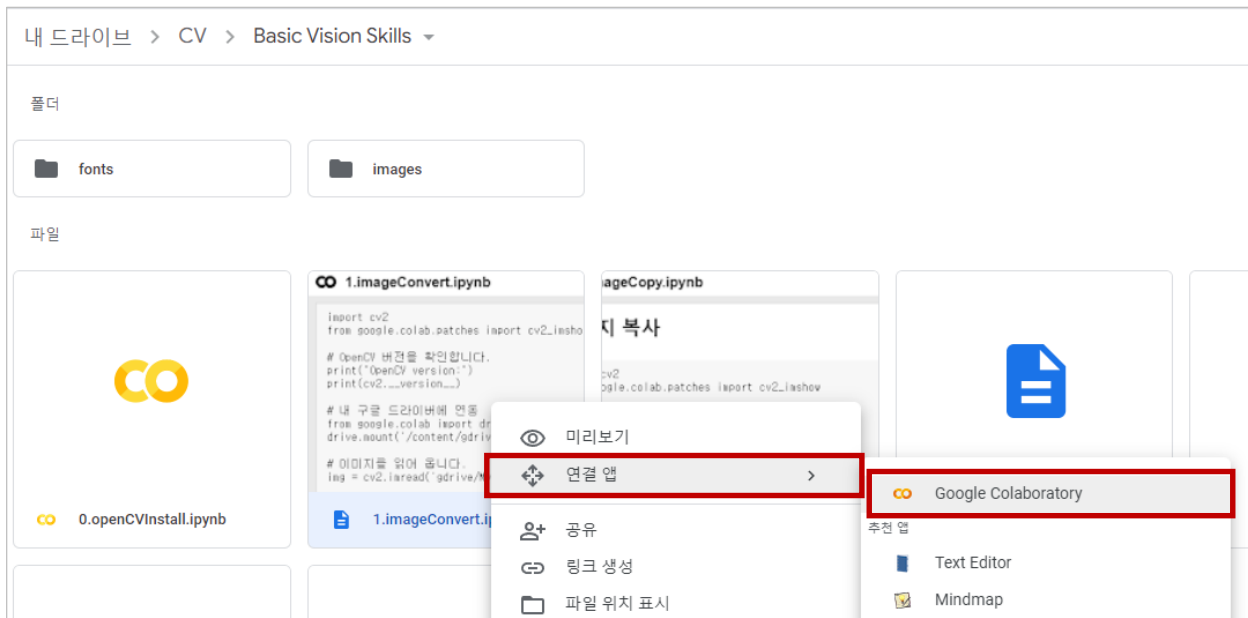
이미지를 읽어 온다.

이미지의 컬러를 변경한다.

이미지를 화면에 출력한다.

1 imageConvert.ipynb 구글 Colab 연결

- 아래의 그림의 Basic Vision Skills 폴더에서 1.imageConvert.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



1 imageConvert.ipynb 구글 Colab 연결

■ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



2 OpenCV 버전 확인

■ 다음은 필요한 패키지를 읽어오고, openCV 설치 버전을 확인해 보자.

🔄 openCV를 사용하기 위해 cv2 패키지 등 **필요한 모듈**을 **읽어온다**.

```
import cv2
from google.colab.patches import cv2_imshow
```

🔄 **openCV**를 설치 **버전**을 **확인**한다.

```
▶ # OpenCV 버전을 확인합니다.
print("OpenCV version:")
print(cv2.__version__)
```

```
📄 OpenCV version:
4.6.0
```

3 구글 Colab 화면에서 내 구글 드라이브 연동

■ 내 구글 드라이브에 연동한다.

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 내 구글 드라이브에 연동
from google.colab import drive

drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

- [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동된다.
- gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

4 원본 이미지 읽기 → 형상(세로, 가로, 채널)

■ 다음은 cv2.imread() 함수로 원본 이미지를 읽어온다.

🔄 이미지를 읽어 올 때 컬러 영상으로 읽어온다.

```
1 # 이미지를 읽어 옵니다.  
2  
3 img = cv2.imread('gdrive/My Drive/CV/Basic Vision Skills/images/rear_garden.PNG', cv2.IMREAD_COLOR)  
4 print(img.shape)      # (496, 819, 3) - 원본 이미지 형상
```

📄 (496, 819, 3)

- 원본 이미지의 형상이 (496, 819, 3)인 것을 알 수 있다(형상은 세로,가로,채널 순이다).
- cv2.imread() 함수는 이미지 파일을 읽을 때 사용하는 함수이다.

이미지 파일은 Numpy array 형태의 숫자 값들이 넘어오고
이 숫자가 해당 위치에서의 색을 의미 한다.

5 원본 이미지 그레이 컬러 변경 → 형상(세로, 가로)

■ 다음은 원본 이미지를 cv2.cvtColor() 함수로 그레이(GRAY) 컬러로 변경한다.

🔄 즉 BGR → GRAY 컬러로 변경한다.

```
1  # 이미지를 변환(BGR to GRAY)
2
3  gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
4  print(gray.shape)      # (496, 819) - 그레이 이미지 형상
```

📄 (496, 819)

- 원본 이미지를 그레이 컬러로 변경 후 형상이 (496, 819)인 것을 알 수 있다.
(형상은 세로, 가로 순이다)

6 원본 이미지 화면에 출력

■ 다음은 원본 이미지(BGR 컬러)를 화면에 출력한다.

```
1 # 원본 이미지(BGR 컬러)를 출력  
2 cv2.imshow('img')
```



7 그레이 컬러 이미지 화면에 출력

■ 다음은 변경 이미지(GRAY 컬러)를 화면에 출력한다.

```
1 # 변경 이미지(GRAY 컬러)를 출력  
2 cv2.imshow('gray')
```



imread() : 이미지 읽기

(참고) cv2.imread()

이미지를 읽을 때는 **imread()** 메서드를 사용하는데 **인수로 파일이름**과 함께 **flag**를 넣을 수 있다.

cv2.IMREAD_COLOR

이미지 파일을 **컬러**로 읽어 들인다.
(투명한 부분은 무시되며, flag 디폴트 값이다)

cv2.IMREAD_GRAYSCALE

이미지 파일을 **그레이스케일** 컬러로 읽어 들인다.
(실제 이미지 처리시 중간단계로 많이 사용한다)

cv2.IMREAD_UNCHANGED

이미지 파일을 **알파 채널**까지 포함하여 읽어 들인다.

OpenCV : 이미지 복사

2.imageCopy.ipynb : cv2로 이미지 복사, 색 공간 변경(BGR, RGB)

OpenCV 패키지를 통해 아래의 기능을 수행해 보자.

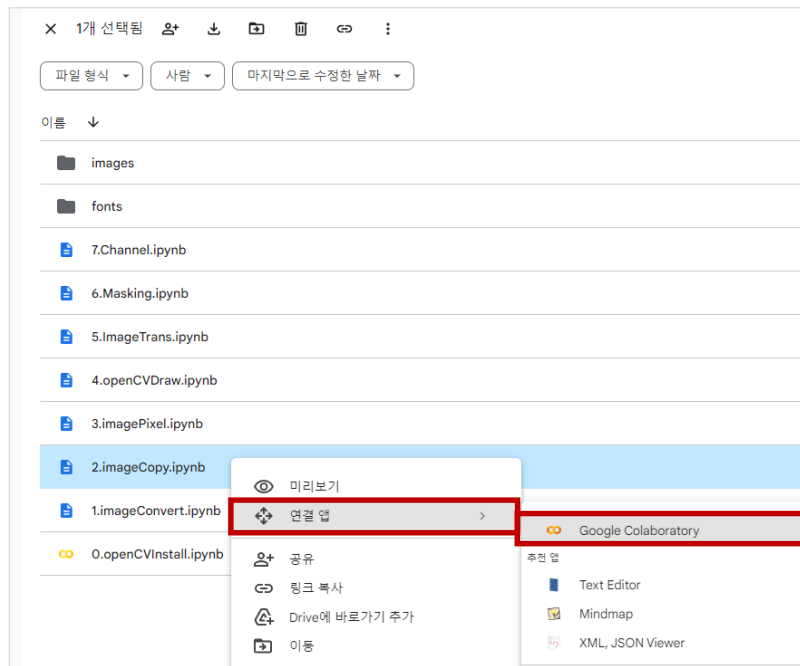
이미지를 읽어 온다.

이미지의 복사한다.

이미지의 색 공간을 변경 한다.

1 imageCopy.ipynb 구글 Colab 연결

- 아래의 그림의 Basic Vision Skills 폴더에서 2.imageCopy.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



1 imageCopy.ipynb 구글 Colab 연결

아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



2 OpenCV 버전 확인

■ openCV를 사용하기 위해 필요한 모듈을 읽고, openCV 버전을 확인해 보자.

🔄 openCV를 사용하기 위해 **cv2 패키지**를 읽어온다.

```
1 # 필요한 모듈 읽어오기
2
3 import cv2
4 from google.colab.patches import cv2_imshow
```

🔄 openCV를 설치 **버전**을 **확인**한다.

```
1 # OpenCV 버전을 확인합니다
2
3 print("OpenCV version:")
4 print(cv2.__version__)
```

📄 OpenCV version:
4.7.0

3 구글 Colab 화면에서 내 구글 드라이브 연동

■ 내 구글 드라이브에 연동한다.

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



- [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동된다.
- gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

3 구글 Colab 화면에서 내 구글 드라이브 연동

[Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용]

아래 화면을 참고하여 gdrive에 연동한다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

[아니오](#) [Google Drive에 연결](#)

Google 계정으로 로그인

계정 선택

Google Drive for desktop(으)로 이동

장원중
wjang2040@gmail.com

다른 계정 사용

계속 진행하기 위해 Google에서 내 이름, 이메일 주소, 언어 환경 설정, 프로필 사진을 Google Drive for desktop(와) 공유합니다. 앱을 사용하기 전에 Google Drive for desktop의 [개인정보처리방침](#) 및 서비스 약관을 검토하세요.

한국어 | 도움말 | 개인정보처리방침 | 약관

구글 드라이브 접근 허용

Google 계정으로 로그인

Google Drive for desktop에서 내 Google 계정에 액세스하려고 합니다.

wjang2040@gmail.com

어떻게 하면 Google Drive for desktop에서 내 작업을 할 수 있습니까?

- Google Drive 파일 보기, 수정, 생성, 삭제
- Google 포토의 사진, 동영상, 앨범을 봅니다.
- 모바일 클라이언트 구성 및 실행 정보 가져오기
- 프로필 및 연락처와 같은 Google 사용자 정보 조회
- Google Drive 파일 작업 기록 조회
- Google Drive 문서 보기, 수정, 생성, 삭제

Google Drive for desktop 앱을 신뢰할 수 있는지 확인

(제한된 정보가 이 사이트 또는 앱과 공유될 수 있습니다. 언제든지 Google 계정에서 액세스 권한을 확인하고 삭제할 수 있습니다.)

Google이 데이터를 안전하게 공유하는 방법을 알아보세요.

Google Drive for desktop의 [개인정보처리방침](#) 및 서비스 약관을 확인하세요.

[취소](#) [허용](#)

한국어 | 도움말 | 개인정보처리방침 | 약관

4 원본 이미지 읽기 → 형상(세로, 가로, 채널)

■ 다음은 cv2.imread() 함수로 원본 이미지를 읽어온다.

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.

```
1 # PNG 이미지를 읽어 온다.  
2  
3 img = cv2.imread("gdrive/My Drive/CV/Basic Vision Skills/images/bukchon.PNG", cv2.IMREAD_COLOR)  
4 print("shape: {}".format(img.shape)) # 형상 - (높이, 가로, 채널)  
5 print("width: {} pixels".format(img.shape[1])) # 넓이  
6 print("height: {} pixels".format(img.shape[0])) # 높이  
7 print("channels: {}".format(img.shape[2])) # 채널
```

```
shape: (496, 882, 3)  
width: 882 pixels  
height: 496 pixels  
channels: 3
```

위 결과에서 원본 이미지의 형상은 (496, 882, 3)인 것을 알 수 있다.

5 원본 이미지 화면 출력

■ 다음은 cv2_imshow() 함수로 원본 이미지를 화면에 출력한다.

🔄 원본 이미지(PNG 형식)를 화면에 출력한다.

```
1 # 원본 이미지를 출력  
2 cv2_imshow(img)
```



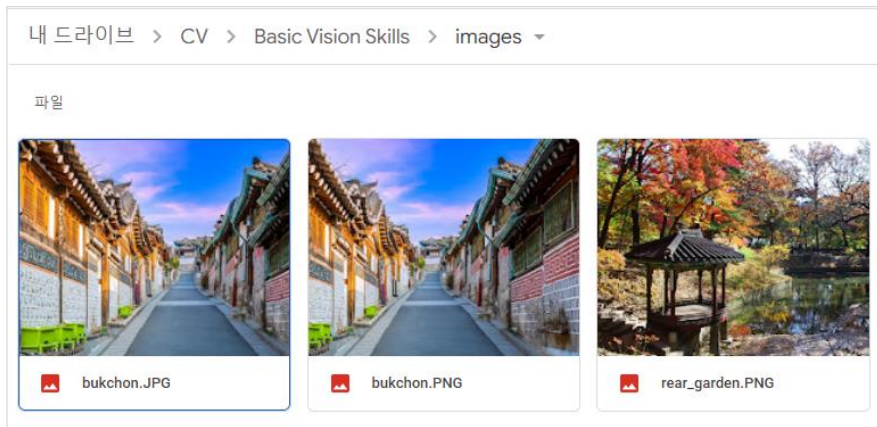
6 원본 이미지(PNG) → JPG 형식으로 저장

- 다음은 cv2.imwrite() 함수로 원본 이미지를 JPG 형식으로 복사하여 저장한다.

```
1 # 원본 이미지를 JPG 형식 복사하여 저장한다.  
2 cv2.imwrite("gdrive/My Drive/CV/Basic Vision Skills/images/bukchon.JPG", img)
```

True

- 🔄 cv2.imwrite() 함수를 활용해 PNG 형식을 JPG 형식으로 복사하여 저장된 것을 볼 수 있다.



7 JPG 이미지 화면 출력

■ 다음은 cv2_imshow() 함수로 **JPG 이미지**를 화면에 출력한다.

🔄 이미지(JPG 형식)를 화면에 출력한다.

```
1 # JPG 이미지를 화면에 출력한다.  
2 cv2_imshow(img2)
```



8 원본 이미지 색 공간 변경(BGR → RGB)

■ 다음은 원본 이미지(BGR)의 각 채널을 분리하여 RGB 색 공간으로 변경한다.

🟢 원본 이미지 각 채널을 분리하여 b, g, r 변수에 저장한다.

```
1 # 원본 이미지의 색 공간 변경 : r, g, b 변수를 조합하여 RGB 색 공간으로 변경
2
3 # 각 채널을 분리
4 b, g, r = cv2.split(img)
5
6 # b, r을 서로 바꿔서 Merge
7 img3_rgb = cv2.merge([r, g, b])
8
9 # 이미지 형상 출력
10 print(img3_rgb.shape) # 형상 - (496, 882, 3)
```

📄 (496, 882, 3)

위 코드에서 원본 이미지의 각 채널을 분리하여 b, g, r 변수에 각 채널 값을 저장한다.

위 결과에서 색 공간 변경 이미지의 형상은 (496, 882, 3)인 것을 알 수 있다.

9 RGB 색 공간으로 변경된 이미지 화면 출력

■ 다음은 RGB 색 공간으로 변경된 이미지를 화면에 출력한다.

```
1 # 색 공간 변경 이미지를 출력한다.  
2 cv2_imshow(img3_rgb)
```



BGR vs. RGB 색 공간 이미지 화면 출력

다음은 BGR vs. RGB 색 공간을 비교해 보자.



BGR 이미지



RGB 이미지



10 원본 이미지 색 공간 변경 → B, G, R 조합

■ 다음은 원본 이미지의 각 채널을 B, G, R 색 공간으로 조합한다.

🔄 원본 이미지 각 채널의 b, g, r 변수를 조합하여 BGR 색 공간으로 변경한다.



```
1 # 원본 이미지의 색 공간 변경 : b, g, r 변수를 조합하여 BGR 색 공간으로 변경
2
3 # b, r을 서로 바꿔서 Merge
4 img3_bgr = cv2.merge([b, g, r])
5
6 # 이미지 형상 출력
7 print(img3_bgr.shape) # 형상 - (496, 882, 3)
```

📄 (496, 882, 3)

위 결과에서 색 공간 변경 이미지의 형상은 (496, 882, 3)인 것을 알 수 있다.

11 BGR 색 공간으로 변경된 이미지 화면 출력

다음은 BGR 색 공간으로 변경된 이미지를 화면에 출력한다.

```
1 # BGR 색 공간 변경 이미지를 출력한다.  
2 cv2_imshow(img3_bgr)
```



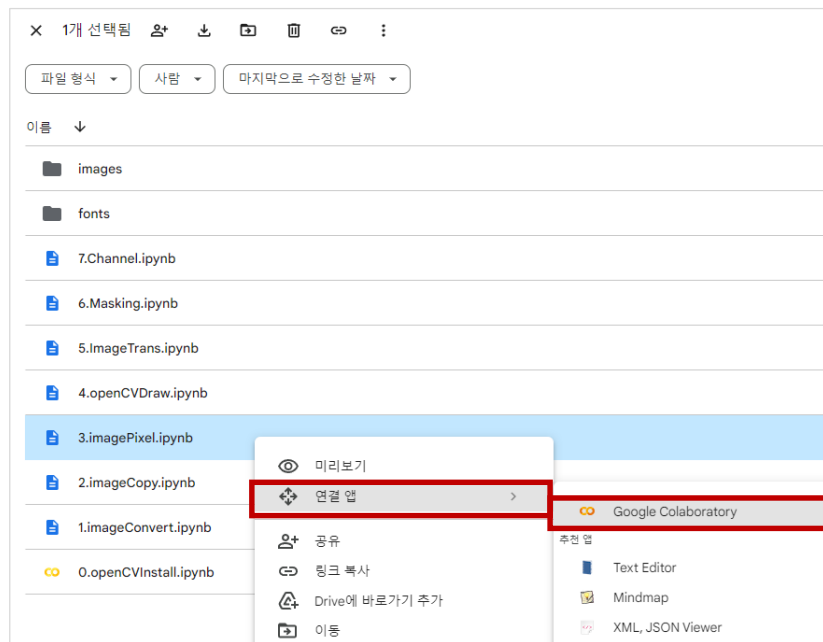
3.imagePixel.ipynb : 픽셀, 해상도 이해

이미지에서 화소(pixel)의 의미를 살펴보자.

- 하나의 픽셀은 Color에 대한 정보를 가지고 있으며, 이러한 수많은 픽셀들이 모여서 사진과 같은 이미지를 이룬다.
- 픽셀은 이미지의 해상도를 나타내는데 사용되며 해상도는 바로 픽셀의 개수를 의미한다.
- 따라서, 해상도는 이미지의 크기와 같은 의미라고 봐도 될 것이다.
- 예를 들어 이미지의 해상도가 1024 x 768 이라고 한다면, 가로(넓이)가 1024개, 세로(높이)가 768개의 픽셀로 이루어진 이미지를 의미한다.

1 imagePixel.ipynb 구글 Colab 연결

- 아래의 그림의 Basic Vision Skills 폴더에서 3.imagePixel.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭한다.



1 imagePixel.ipynb 구글 Colab 연결

■ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



2 OpenCV 버전 확인

■ openCV를 사용하기 위해 필요한 모듈을 읽고, openCV 버전을 확인해 보자.

🔄 openCV를 사용하기 위해 **cv2 패키지**를 읽어온다.

```
1 # 필요한 모듈 읽어오기
2
3 import cv2
4 from google.colab.patches import cv2_imshow
```

🔄 openCV를 설치 **버전**을 **확인**한다.

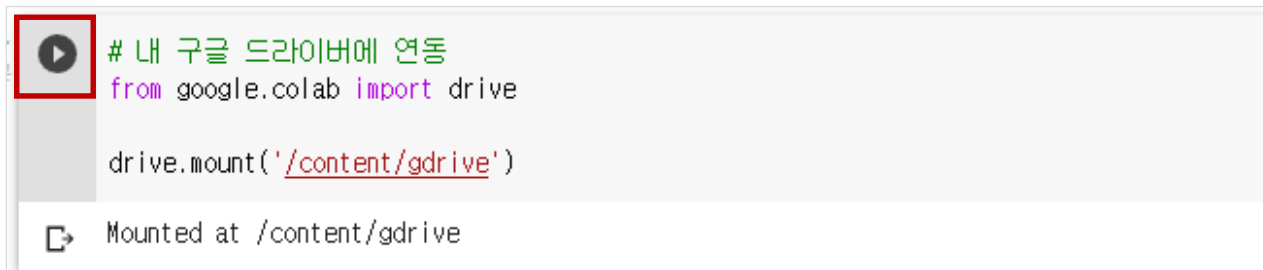
```
1 # OpenCV 버전을 확인합니다
2
3 print("OpenCV version:")
4 print(cv2.__version__)
```

📄 OpenCV version:
4.7.0

3 구글 Colab 화면에서 내 구글 드라이브 연동

■ 내 구글 드라이브에 연동한다.

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 내 구글 드라이브에 연동
from google.colab import drive

drive.mount('/content/gdrive')
```

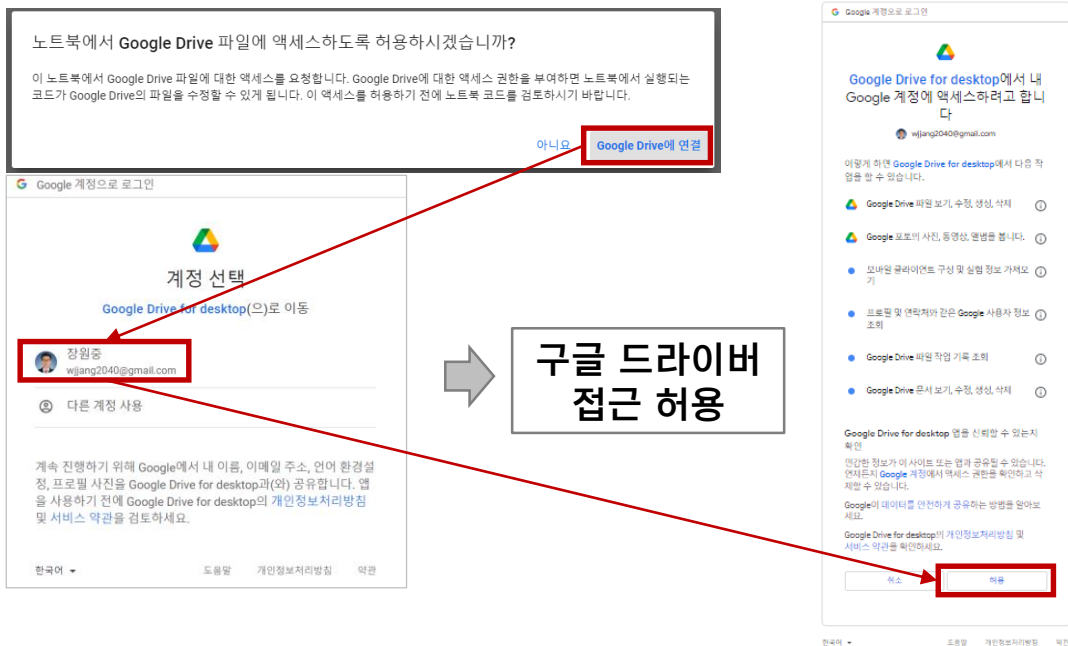
Mounted at /content/gdrive

- [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동된다.
- gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

3 구글 Colab 화면에서 내 구글 드라이브 연동

■ [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]

아래 화면을 참고하여 gdrive에 연동한다.



4 원본 이미지 읽기 → 형상(세로, 가로, 채널)

■ 다음은 cv2.imread() 함수로 원본 이미지를 읽어온다.

🔄 원본 이미지를 읽어올 때 컬러로 읽어온다.

```
1 # 이미지를 읽어 온다.  
2 img = cv2.imread('gdrive/My Drive/CV/Basic Vision Skills/images/rear_garden.PNG', cv2.IMREAD_COLOR)  
3 print(img.shape) # 형상 - (496, 819, 3)  
4 print("width: {} pixels".format(img.shape[1])) # 넓이  
5 print("height: {} pixels".format(img.shape[0])) # 높이  
6 print("channels: {}".format(img.shape[2])) # 채널
```

```
↳ (496, 819, 3)  
width: 819 pixels  
height: 496 pixels  
channels: 3
```

위 결과에서 원본 이미지의 형상은 (496, 819, 3)인 것을 알 수 있다.

5 원본 이미지 화면 출력

■ 다음은 cv2_imshow() 함수로 원본 이미지를 화면에 출력한다.

🔄 원본 이미지(PNG 형식)를 화면에 출력한다.

```
1 # 원본 이미지를 출력  
2 cv2_imshow(img)
```



6 원본 이미지[0, 0] 위치의 Color 정보 확인

■ 원본 이미지의 [0, 0] 위치는 어떠한 Color 정보를 가지고 있는지 알아보자.

- 🔄 원본 이미지의 색상을 정할 때 OpenCV 이미지의 기본 채널 순서는 파랑-녹색-빨간(BGR) 방식을 사용한다.

```
1 # 색상을 정할 때 OpenCV 이미지의 기본 채널 순서는 파랑-녹색-빨간(BGR) 방식을 사용한다.  
2  
3 # 이미지의 특정 위치 = 픽셀(Pixel), Pixel(BGR)  
4 (b, g, r) = img[0, 0]  
5 print("Pixel at (0, 0) - Red: {}, Green: {}, Blue: {}".format(r, g, b))
```

👉 Pixel at (0, 0) - Red: 8, Green: 20, Blue: 18

[0, 0] 위치의 픽셀 값은 R=8, G=20, B=18 인 색상 정보를 가지고 있음을 확인 할 수 있다.

7 이미지의 특정한 영역 출력

■ 원본 이미지에서 특정한 범위만 지정하여 화면에 출력해 보자.

🔄 원본 이미지의 [50:100, 50:100] 영역만을 화면에 출력한다.

```
1 # 이미지에서 특정한 범위 img[50:100, 50:100] 만 지정하여 출력
2
3 ROI = img[50:100, 50:100]
4 cv2_imshow(ROI)
```



위 결과에서 원본 이미지의 특정 영역만 출력된 것을 볼 수 있다.

8 이미지의 특정한 영역에 빨강색으로 변경

■ 원본 이미지의 특정 범위 [50:100, 50:100] 영역에 빨강색으로 변경해 이미지를 출력해 보자.

🔄 원본 이미지의 [50:100, 50:100] 영역에 (0,0,255) 픽셀값을 설정한다.

```
1 # 이미지에서 특정한 범위 img[50:100, 50:100] 만 지정하여 색상을 빨간색으로 변경
2 img[50:100, 50:100] = (0, 0, 255)
3 cv2.imshow('img')
```

